

CSCE 650 VIRTUAL REALITY · MODULE 05 · LECTURE NOTES

Tracking

Knowing where the head is, continuously and without drift.

MODULE OUTCOMES

- Explain what an IMU measures and why it drifts.
- Explain sensor fusion and the role of each sensor.
- Compare inside-out and outside-in tracking.
- Explain prediction and why it is necessary.
- Explain tracking failure modes and how to handle them.

1 The inertial measurement unit

Sensor	Measures	Rate	Problem
Gyroscope	Angular velocity about three axes.	~1000 Hz	Must be integrated once to get orientation , so any bias accumulates linearly — degrees per minute of drift.
Accelerometer	Linear acceleration along three axes, <i>including gravity</i> .	~1000 Hz	Must be integrated twice to get position , so bias accumulates quadratically. Gravity must also be separated out perfectly, and it is an order of magnitude larger than the signal of interest.
Magnetometer	The local magnetic field direction.	~100 Hz	Gives an absolute heading reference — and is distorted by ferrous metal, speaker magnets, and electrical wiring, which is everything in an ordinary room.

DOUBLE INTEGRATION IS WHY POSITION DRIFTS SO QUICKLY

Orientation requires integrating angular velocity once. A constant bias of b degrees per second produces an error of $b \cdot t$ — linear in time, and recoverable with an occasional absolute reference.

Position requires integrating acceleration twice. A constant bias b produces an error of $\frac{1}{2}bt^2$ — quadratic. And before the first integration, gravity must be subtracted, which requires knowing the orientation precisely, which has its own error.

Concretely: an accelerometer bias of 0.01 m/s^2 — a very good sensor — gives half a metre of position error after ten seconds, and two metres after twenty.

So purely inertial position tracking is hopeless on any useful timescale, and every system corrects it against an optical reference. **The IMU supplies the rate; the cameras supply the truth.** This division is the whole architecture of modern tracking.

2 Sensor fusion

	IMU	Optical
Rate	~1000 Hz	30–90 Hz
Latency	Under a millisecond	Tens of milliseconds — exposure, transfer, and feature extraction all cost time.
Noise	Smooth; little jitter.	Noisy, and subject to outright failure.
Drift	Accumulates without bound.	None — it is an absolute reference.
Tells you	How you are moving <i>right now</i> .	Where you actually are, <i>somewhat late</i> .

THE FUSION PRINCIPLE

The two sensors are exactly complementary: each one's weakness is the other's strength. This is unusually clean, and it is why tracking works as well as it does.

Use the IMU for high-frequency motion. Over tens of milliseconds it is fast, smooth, and accurate, and it is the only thing that can respond quickly enough for VR.

Use the optical estimate to correct low-frequency drift. It arrives late and noisily, and it never drifts, so it anchors the integration.

A complementary filter is the simple implementation — a high-pass on the IMU and a low-pass on the optical estimate, summed. **An extended or unscented Kalman filter is the production answer**, estimating the sensor biases as part of its state so that the integration improves over time rather than merely being corrected.

The fused result is substantially better than either sensor alone, which is the standard outcome for genuinely complementary sensors and the reason every tracking system in existence has this shape.

	Outside-in	Inside-out
Cameras	Fixed in the room — base stations or sensors.	Mounted on the headset.
Tracks	Markers or emitters on the headset and controllers.	Natural features in the room — essentially SLAM.
Accuracy	Higher, and better conditioned — the geometry is known and fixed.	Good, and dependent on the room having trackable features.
Setup	Base stations must be mounted and calibrated.	None. Put the headset on.
Play space	Bounded by the base stations' coverage.	Unbounded in principle.
Characteristic failure	Occlusion — your own body between the station and a controller, which happens constantly.	Featureless or repetitive surfaces, and poor light.

Inside-out won on convenience, decisively, once the computer-vision quality was adequate. Its failure modes are different rather than absent: a large blank white wall gives it nothing to track against, and a featureless room is genuinely difficult in a way that is invisible until you are in one.

3 Prediction

YOU MUST RENDER FOR A FUTURE POSE

By the time a rendered frame's photons leave the display, the pose it was rendered from is **tens of milliseconds old** — the full motion-to-photon pipeline of Module 06.

So the runtime predicts. It extrapolates the head's trajectory to the expected moment of display, and that predicted pose is what you render from. You are never rendering where the head is.

Short-horizon prediction is remarkably accurate, because a head is a mass on a neck and cannot change direction instantaneously. Angular velocity is strongly autocorrelated over short intervals, so **20 to 40 ms ahead is reliable** — reliable enough that the residual error is below perceptual threshold for most motion.

Beyond that it degrades, and the characteristic failure is overshoot at the moment the head changes direction: the prediction continues the old motion briefly, and the world appears to swim. **This is the fundamental limit on how much latency prediction can conceal**, and it is why prediction supplements rather than replaces actually reducing latency.

- **Ask the runtime for the pose at the predicted display time**, which is what OpenXR's API is built around — you pass the target display time and receive the predicted pose. Asking for 'the current pose' is the wrong question.
- **Use one predicted pose for the whole frame**, for both eyes and all objects. Querying the pose separately per object produces objects rendered from slightly different viewpoints, which is inconsistent in a way that is hard to diagnose.
- **Late latching** defers reading the pose until as late as possible — after the CPU-side frame setup, immediately before the GPU needs it — which shortens the prediction horizon and therefore reduces the prediction error.
- **Do not predict further ahead than necessary.** A longer horizon means a larger error, and timewarp (Module 06) corrects residual error more accurately than prediction avoids it, because timewarp uses a pose measured rather than extrapolated.

TRACKING LOSS MUST BE HANDLED EXPLICITLY

Tracking will fail. A hand over a camera, a user who walks into a dark corner, direct sunlight saturating a sensor, a controller held behind the back. This is a normal operating condition, not an exceptional one.

Snapping the view to a default pose is the worst available response. An instantaneous, unrequested change of viewpoint is maximally disorienting and is a reliable way to make someone ill (Module 07). It is also, unfortunately, the default behaviour of naive code that simply uses whatever pose it last received.

Degrade gracefully instead: fall back to orientation-only tracking, which the IMU can sustain for a while; hold the last known position rather than jumping; and if the loss persists, fade the world to black rather than showing something wrong.

And tell the user. A clear indication that tracking has been lost is far better than a world that has begun behaving inexplicably — people tolerate a known failure and are disturbed by an unexplained one.

- **Featureless surfaces.** A large blank white wall offers inside-out tracking nothing to lock onto.
- **Poor or changing light.** Cameras need photons; a dark room is hard, and a room with flickering or rapidly changing light is harder.
- **Repetitive texture.** A regularly tiled floor or a brick wall produces false feature matches, which can cause the estimated position to jump by exactly one tile.
- **Moving environments.** In a car, train, or boat, the accelerometer reports the vehicle's acceleration as the head's, and the optical system sees a stationary interior. The two disagree permanently. **This is a genuinely unsolved case** and the reason using VR on a train is unpleasant.
- **Controller occlusion** behind the body or outside the headset cameras' field of view — which with inside-out tracking happens whenever the user puts their hands at their sides, draws a bow, or reaches behind their head. Predicted controller poses bridge short gaps; long ones cannot be hidden.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
LaValle — Virtual Reality, Chapter 9 (free)	Tracking in full: IMU models, drift, filtering, and the mathematics of fusion. Written by someone who built the Oculus tracker, and the best free source for this module.
Stanford EE267 — IMU and pose tracking assignment (free)	Implement a complementary filter on real IMU data. The drift is far more convincing when you have watched it accumulate.
Madgwick — An efficient orientation filter for IMUs (free)	A widely used practical fusion filter, with source. A good implementation target.
OpenXR — space and pose prediction (free)	The predicted-display-time API of §3, and the pose validity flags you need for tracking loss.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Log raw gyroscope data from any device and integrate it. Plot the orientation drift over five minutes.
- 2 Integrate accelerometer data twice to get position and plot the drift over ten seconds. Compare the growth rate against the gyroscope.
- 3 Implement a complementary filter fusing a drifting fast signal with a noisy slow one, and show the fused result beats both.

- 4 In a headset, cover a camera and observe what happens. Then try a blank wall, a dark room, and a tiled floor.
- 5 Measure the practical play-space limits of an inside-out system by walking until tracking degrades.
- 6 Request the pose at the predicted display time and at the current time, and compare them during head motion. Report the difference in degrees.
- 7 Deliberately render from an un-predicted pose and describe the sensation.
- 8 Implement graceful tracking-loss handling: orientation-only fallback, position hold, and a user-visible indicator. Test by covering a camera.
- 9 Use a headset in a moving vehicle, briefly, and document why it fails.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 What do a gyroscope and accelerometer measure, and which measures position?
- 2 Why does position drift faster than orientation? Give the growth rates.
- 3 Compare IMU and optical tracking on rate, latency, noise, and drift.
- 4 State the fusion principle and name the usual filters.
- 5 Compare inside-out and outside-in on six axes.
- 6 Why must you render from a predicted pose, and why is prediction accurate?
- 7 What limits the prediction horizon, and what does overshoot feel like?
- 8 Give four rules for using prediction correctly.
- 9 Why is snapping to a default pose the worst response to tracking loss?
- 10 Name five causes of tracking failure.